Chapter 5: Interrupts and Device Drivers

xv6 RISC-V Study Notes

Based on Cox, Kaashoek & Morris — xv6: a simple, Unix-like teaching OS *(rev. 3, 2022)*

1 Big Picture: What Problem Does This Chapter Solve?

1.1 The Core Challenge

Hardware devices operate *asynchronously*: a keyboard fires an interrupt the moment a key is pressed; a disk signals completion unpredictably. The CPU cannot simply poll every device in a tight loop — it would waste all its time checking devices that rarely have work to do.

The operating system needs a principled way to:

- **Configure** hardware to generate interrupts at the right moments.
- **Dispatch** incoming interrupts to the correct driver without losing data.
- **Decouple** the slow device from the fast process so neither has to wait on the other.
- **Protect** shared driver data structures from concurrent access by multiple CPUs and interrupt handlers.

Chapter 5 answers these questions using the UART console as a worked example, then generalises to timer interrupts and the broader landscape of device I/O.

1.2 What is a Driver?

A **driver** is kernel code that owns a single device. It has three jobs:

1. **Configure** the device hardware (memory-mapped registers, interrupt enable bits).
2. **Issue commands** to the device and manage I/O requests from user processes.
3. **Handle interrupts** that the device raises when an operation completes or new data arrives.

Drivers are tricky because they execute *concurrently with the device hardware* and must reason carefully about shared state, interrupt timing, and multiple CPUs.

1.3 The Two Halves of Every Driver

Top Half (Process Context)

Called from a process context (e.g., via a `read` or `write` system call).
Starts an I/O operation (writes to device registers), then **sleeps** waiting for completion.
Example: `consoleread` waiting for a full line of input.

Bottom Half (Interrupt Context)

Called from interrupt context (the CPU's interrupt handler fires when the device is done).
Collects completed data, **wakes up** the sleeping process, and queues the next operation.
Example: `uartintr` reading a received character and calling `consoleintr`.

Foundational Concept

Why split into two halves? A process cannot spin-wait in the kernel for a slow device — that would monopolise a CPU. By sleeping in the top half and using the interrupt to signal completion, the kernel frees the CPU to run other processes while the device is busy. This is the core pattern behind all I/O concurrency.

Section Summary

Drivers own devices. Every driver splits work into a *top half* (starts I/O, sleeps) and a *bottom half* (interrupt handler, wakes up sleeping processes). This split is what allows the CPU to remain productive while slow hardware finishes its work.

2 Core Mechanisms

2.1 Memory-Mapped I/O (MMIO)

Hardware devices appear to software as a set of *memory-mapped registers*: physical addresses that, when read or written, talk to the device rather than RAM.

- The UART device in xv6 is a **16550 chip** (emulated by QEMU).
- Its registers are mapped starting at physical address `0x10000000` (`UART0`).
- Key registers: `RHR` (receive holding register — read incoming bytes), `THR` (transmit holding register — write outgoing bytes), `LSR` (line status — flags whether data is waiting or the transmitter is ready).
- A regular load/store to these addresses interacts with the device; no special instructions are needed.

2.2 The PLIC: Platform-Level Interrupt Controller

Multiple devices can raise interrupts. The CPU needs to know *which* device interrupted.

- The **PLIC** (Platform-Level Interrupt Controller) sits between all external devices and each CPU core.
- When a device raises an interrupt, the PLIC holds it and signals the CPU.
- `devintr()` asks the PLIC which device interrupted (`kernel/trap.c:187`) and dispatches to the appropriate driver interrupt handler.
- After the interrupt is handled, the driver clears the interrupt at the PLIC.

2.3 The CLINT: Core-Local Interruptor (Timer Only)

Timer interrupts use a separate mechanism from all other interrupts.

- Each RISC-V CPU core has a **CLINT** (Core-Local Interruptor) that generates periodic timer interrupts.
- Timer interrupts are delivered in **machine mode** — a more privileged level than supervisor mode (where the kernel normally runs).
- Because machine-mode code runs without paging and has its own register set, xv6 handles timer interrupts in a minimal machine-mode handler (`timervec`) that simply re-raises them as a **software interrupt** visible to supervisor mode.

Section Summary

Devices expose themselves via memory-mapped registers. The PLIC arbitrates external device interrupts, telling the CPU which device fired. The CLINT handles per-CPU timer interrupts via machine mode, which xv6 bounces through as software interrupts into the normal supervisor-mode trap path.

3 Hardware vs. Kernel Responsibilities

Hardware Responsibilities

When a device interrupt arrives, RISC-V hardware automatically:

1. Checks `sstatus.SIE`; if clear, defers the interrupt.
2. Clears `sstatus.SIE` (disables further interrupts).
3. Saves `pc` to `sepc`.
4. Records current mode in `sstatus.SPP`.
5. Sets `scause` to indicate an external interrupt.
6. Switches to supervisor mode.
7. Jumps to `stvec` (either `kernelvec` or `uservec` depending on context).

The UART hardware additionally:

- Puts received bytes into its internal FIFO and asserts an interrupt line to the PLIC.
- Asserts an interrupt line when the transmit FIFO is ready for more data.

Kernel / Driver Responsibilities

The kernel (trap handler + driver) must then:

1. Save registers (via `uservec`/`kernelvec` as in Ch. 4).
2. Call `devintr()` which queries the PLIC to identify the interrupting device.
3. Dispatch to the device's interrupt handler (e.g., `uartintr()`).
4. Read/write the device's MMIO registers to collect data or push new data.
5. Signal the PLIC that the interrupt has been handled.
6. Wake up any sleeping process waiting on this I/O.
7. Re-enable interrupts (`sstatus.SIE`) before returning.
8. Restore registers and execute `sret`.

Section Summary

The hardware-kernel split from Chapter 4 applies directly to device interrupts: RISC-V saves the PC, switches mode, and jumps to `stvec`; the kernel saves all registers, identifies the device via the PLIC, runs the driver's interrupt handler, and restores everything on return.

4 Console Input: Full Lifecycle

4.1 Setup

`consoleinit()` (`kernel/console.c:182`) configures the UART to generate:

- A **receive interrupt** every time the UART receives a byte of input.

- A **transmit-complete interrupt** every time the UART finishes sending a byte of output.

Top half — Process waits for input:

1. A shell process calls `read()`; this arrives in the kernel at `consoleread()` (`kernel/console.c:80`).
2. `consoleread` checks `cons.buf`. If no full line is available, it calls `sleep()` and yields the CPU.
3. The process is now blocked. The CPU runs other work.

Bottom half — Key press arrives:

4. User presses a key; UART hardware places the byte in its FIFO and signals the PLIC.
5. RISC-V trap fires. `devintr()` is called; it queries the PLIC and identifies the UART.
6. `devintr()` calls `uartintr()` (`kernel/uart.c:176`).
7. `uartintr()` reads all available bytes from `RHR` and passes each to `consoleintr()` (`kernel/console.c:136`).
8. `consoleintr()` appends each character to `cons.buf`, handling special characters (backspace, Ctrl-U) as it goes.
9. When a newline arrives, `consoleintr()` calls `wakeup()` to wake the sleeping `consoleread`.

Top half resumes:

10. `consoleread` wakes up, copies the completed line from `cons.buf` to user space, and returns to the shell.

Implementation Detail

`uartintr()` does *not* wait for characters — it reads whatever is present in the FIFO right now and returns immediately. Future characters will trigger a new interrupt. The driver never spins on the hardware.

5 Console Output: Full Lifecycle

Top half — Process writes output:

1. A process calls `write()` on the console; this arrives at `uartputc()` (`kernel/uart.c:87`).
2. `uartputc()` appends the character to the circular output buffer `uart_tx_buf`.
3. It calls `uartstart()` to start transmission (if the UART is not already sending).
4. `uartstart()` checks if the UART's transmit register is free; if so, it writes the first buffered byte to `THR` and returns.
5. `uartputc()` returns immediately. The process continues — it does *not* wait for the UART to finish. (The only exception: if `uart_tx_buf` is full, the process sleeps until space is available.)

Bottom half — UART finishes sending:

6. UART hardware finishes sending the byte and raises a *transmit-complete* interrupt.
7. `uartintr()` fires; it calls `uartstart()` again.
8. `uartstart()` pulls the next buffered character, writes it to `THR`, and returns.
9. This repeats for each buffered character until the buffer is empty.

Foundational Concept

I/O Concurrency via Buffering: The output buffer (`uart_tx_buf`) decouples the process from the device. The process deposits bytes into the buffer and keeps running; the interrupt handler drains the buffer at whatever rate the UART can handle. This pattern — a software buffer bridging a fast producer and a slow consumer — appears throughout OS design (disk caches, network buffers, pipe buffers).

Section Summary

Console input: a process sleeps on `consoleread`, an interrupt wakes it when a line arrives. Console output: `uartputc` buffers immediately and returns; interrupts drain the buffer asynchronously. Both patterns rely on a buffer and a sleep/wakeup pair to decouple the process from the slow device.

6 Concurrency in Drivers

6.1 The Three Concurrency Dangers

Three Concurrency Hazards in the Console Driver

1. **Two processes on different CPUs both call `consoleread`.** Both may manipulate `cons.buf` simultaneously, corrupting its state.
2. **A CPU is inside `consoleread` when a UART interrupt arrives on the *same* CPU.** The interrupt handler (`consoleintr`) also accesses `cons.buf`. Without a lock, the data structure is corrupted by re-entrant access.
3. **A UART interrupt arrives on a *different* CPU while `consoleread` is running.** Two execution contexts on two separate cores simultaneously modify shared data.

All three are guarded by the `cons.lock` spinlock, acquired at the start of `consoleread` and `consoleintr`. Chapter 6 explains how these locks work.

6.2 Interrupt Handler Constraints

Interrupt handlers run in a special context with important restrictions:

- An interrupt handler **cannot know which process is currently running** — it may fire while any (or no) process is scheduled on that CPU.
- Therefore an interrupt handler **must not use the current process's page table** (e.g., it cannot safely call `copyout` with a user address).
- Interrupt handlers should do **as little work as possible** — typically just buffer the data and wake up top-half code to do the real processing.

Foundational Concept

Why interrupt handlers must be lean: Because timer interrupts can fire at *any* moment, including inside other interrupt handlers (unless SIE is cleared), and because the handler cannot safely assume anything about the currently running process, interrupt handlers must limit themselves to operations that are safe in any context: reading MMIO registers, writing to kernel buffers, and signalling sleeping processes via `wakeup()`.

Section Summary

Drivers face three concurrency threats: multi-CPU access, same-CPU re-entrant interrupt delivery, and cross-CPU interrupt delivery. All three are handled with locks. Interrupt handlers are further restricted: they must not assume a particular process context and should defer heavy work to the top half via sleep/wakeup.

7 Timer Interrupts

7.1 Why Timer Interrupts Are Different

Timer interrupts serve a special role: they are the mechanism by which xv6 **preempts** long-running processes and switches to other threads (via `yield()`). Without timer interrupts a CPU-bound process could monopolise a core forever.

Timer Interrupt Peculiarities

- Timer interrupts are delivered in **machine mode** (not supervisor mode).
- Machine mode has its own separate control registers (`mtvec`, `mscratch`, etc.) and runs without paging.
- It is not practical to run normal xv6 kernel C code in machine mode.
- Therefore, xv6 handles timer interrupts in a **pure-assembly machine-mode handler** (`timervec`, `kernel/kernelvec.S:95`) that does the minimum necessary, then re-raises the interrupt as a **software interrupt** (SSIP bit) visible in supervisor mode.
- The supervisor-mode trap path then handles it via the normal `devintr()` dispatch.

7.2 Timer Interrupt Lifecycle

Setup (once at boot, in `start.c` before `main`):

1. Program the **CLINT** to generate an interrupt after a fixed number of clock ticks.
2. Set up a per-CPU *scratch area* (analogous to a machine-mode trapframe) to hold saved registers and the CLINT address.
3. Set `mtvec` to `timervec` and enable machine-mode timer interrupts.

Every timer tick:

4. CLINT fires; CPU enters machine mode at `timervec`.
5. `timervec` saves a few registers to the scratch area.
6. `timervec` programs the CLINT for the *next* timer interrupt.
7. `timervec` sets the **SSIP** bit in `sip` — this requests a supervisor-mode software interrupt.
8. `timervec` restores saved registers and returns from machine mode.
9. CPU takes the pending software interrupt in supervisor mode via the normal trap path (`usertrap` or `kerneltrap`).
10. `devintr()` recognises it as a software interrupt and returns the timer interrupt code.
11. `usertrap`/`kerneltrap` calls `yield()` to switch to another process.

Foundational Concept

The indirection through software interrupt: Since timer interrupts arrive in machine mode and xv6 kernel code lives in supervisor mode, xv6 cannot simply call `yield()` from the machine-mode handler. The machine-mode handler's only job is to schedule the next timer tick and *poke* supervisor mode by raising a software interrupt. The real work (context switching) happens in the well-understood supervisor-mode trap path.

Section Summary

Timer interrupts arrive in machine mode, which xv6 keeps minimal (pure asm, no C). The machine-mode handler reschedules the CLINT and re-delivers the interrupt as a supervisor-mode software interrupt. Supervisor mode then calls `yield()` through the normal trap path to preempt the running process.

8 Foundational vs. Implementation Details

8.1 Foundational Concepts (Must Understand)

Foundational Concepts

1. **The top-half / bottom-half split is the universal driver pattern.** Starting I/O in a process context and completing it in an interrupt context (with a buffer and sleep/wakeup in between) is how every I/O driver in every OS works.
2. **Memory-mapped I/O is how software talks to hardware.** Device registers look like ordinary memory addresses. The OS accesses devices with the same load/store instructions it uses for RAM.
3. **Buffering is what makes I/O concurrency possible.** Without a buffer between the fast process and the slow device, one must wait on the other. A buffer lets both proceed at their own pace.
4. **Interrupt handlers must be context-agnostic.** Because an interrupt can arrive in the middle of any process (or no process), interrupt handlers must not rely on process-specific state. They should do minimal work and delegate to sleeping top-half code.
5. **Timer interrupts are the preemption mechanism.** Without periodic timer interrupts, a runaway or CPU-bound process could permanently monopolise a CPU core.
6. **Machine mode is a separate island from supervisor mode.** RISC-V's three privilege levels (user, supervisor, machine) each have their own trap registers. Timer interrupts at machine mode require their own minimal handler that bounces work back to supervisor mode via software interrupts.

8.2 Implementation Details (Good to Know)

Implementation Detail

- UART0 is mapped at `0x10000000`; register offsets (RHR, THR, LSR, etc.) are defined in `kernel/uart.c:22`.
- `consoleinit()` configures the UART for receive and transmit-complete interrupts (`kernel/console.c:182`).
- `uart_tx_buf` is a circular buffer; `uartputc` sleeps only when the buffer is completely full.
- `uartstart()` is called by both `uartputc` (top half) and `uartintr` (bottom half) to avoid duplicating the "write next byte to THR" logic.
- `cons.buf` is a fixed-size circular buffer; `consoleintr` handles backspace by decrementing the write pointer.
- The PLIC interrupt dispatch happens in `devintr()` at `kernel/trap.c:178`; it reads the PLIC claim register to get the device ID.
- `timervec` saves and restores exactly 3 registers using the scratch area prepared by `start.c`; there is no C code involved in the machine-mode timer handler.
- The SSIP bit in the `sip` CSR is how machine-mode code requests a software interrupt in supervisor mode.
- xv6 is **not** suitable for hard real-time use: interrupts are disabled in parts of the kernel, and the scheduler is simple; worst-case latency is unbounded.

9 Real World Context

9.1 Programmed I/O vs. DMA

	Programmed I/O (xv6)	**DMA (production OS)**
How data moves	CPU reads/writes device registers one byte at a time	Device hardware writes data directly to RAM
CPU involvement	High (CPU in the loop for every byte)	Low (CPU only touched for setup and completion)
Suitability	Simple, slow devices (UART)	Fast devices (disk, NIC, GPU)
xv6 example	UART reads `RHR` per character	Modern disk controllers, network cards

9.2 Interrupts vs. Polling

	Interrupt-driven (xv6)	**Polling**
Mechanism	Device signals CPU when ready	CPU repeatedly checks device status register
Best for	Infrequent, unpredictable events	High-throughput, low-latency devices
Overhead	Context-switch cost per interrupt	Wasted CPU cycles when device is idle
Hybrid	Some drivers switch modes based on load	—

Section Summary

xv6's UART driver uses programmed I/O (byte-at-a-time via MMIO) and interrupt-driven I/O. Production kernels use DMA for bulk data movement (the device fills RAM directly) and may switch between interrupt-driven and polling modes depending on device load. Drivers can account for more code than the core kernel in a production OS.

10 Quick Reference

10.1 Key Functions

Function	Half	Role
`consoleinit`	Setup	Configures UART for interrupts
`consoleread`	Top	Blocks process until a line is in `cons.buf`
`consoleintr`	Bottom	Accumulates characters into `cons.buf`; wakes reader on newline
`uartintr`	Bottom	Reads from UART FIFO; calls `consoleintr` and `uartstart`
`uartputc`	Top	Appends char to `uart_tx_buf`; calls `uartstart`
`uartstart`	Both	Writes next buffered byte to UART THR if UART is ready
`devintr`	Bottom	Queries PLIC; dispatches to UART, disk, or timer handler
`timervec`	Machine-mode	Reprograms CLINT; raises SSIP software interrupt

10.2 Key Hardware Units

Unit	Address / CSR	Role
UART0	`0x10000000`	Serial I/O device (keyboard/display in QEMU)
PLIC	`0x0c000000`	Routes external device interrupts to CPUs
CLINT	`0x02000000`	Generates per-CPU timer interrupts (machine mode)

11 Overall Chapter Summary

Chapter 5 — Complete Summary

Chapter 5 addresses how the OS manages asynchronous hardware devices.

- **Drivers** own devices. They are split into a *top half* (process context, starts I/O, sleeps) and a *bottom half* (interrupt context, completes I/O, wakes process).
- **Memory-mapped I/O** is how the kernel talks to device registers — ordinary loads and stores to special physical addresses.
- **The PLIC** arbitrates which external device interrupted; `devintr()` queries it and dispatches to the right driver.
- **Console input** flows: key press → UART FIFO → PLIC → `uartintr` → `consoleintr` → `cons.buf` → `wakeup(consoleread)`.
- **Console output** flows: `uartputc` → `uart_tx_buf` → `uartstart` → UART THR → transmit-complete interrupt → `uartstart` (next byte).
- **Concurrency** in drivers is dangerous (three distinct hazard scenarios); locks guard shared buffers. Interrupt handlers must be context-agnostic and minimal.
- **Timer interrupts** arrive in machine mode; a minimal asm handler (`timervec`) reschedules the CLINT and bounces a software interrupt into supervisor mode, where `yield()` is called.
- Real-world kernels extend these patterns with DMA, polling, and mixed-mode strategies for high-throughput devices.